PROJECT WALKTHROUGH

I Built a Full UI Automation Framework from Scratch.

No Selenium. No Cypress. Just Java, Playwright & plain English test steps.

Ali Hussein | QA Automation Engineer

Swipe →

PROJECT STRUCTURE & FLOW

Every file has one clear job.

STRUCTURE

```
playwright-uitesting/
  core/
    BrowserFactory
    LoadingPage
    StepProcessor
  handlers/  (19+)
    Navigation
    Button, Insert
    Dropdown, Group
    ...and more
  utils/
    ExcelReader
    WordReport
    PerfReport
  TestRunner.java
```

EXECUTION FLOW

1. **TestRunner**
   Reads all rows from Excel
2. **ExcelReader**
   Parses TC ID, steps, flags
3. **StepProcessor**
   Routes each keyword to handler
4. **Handler (x19+)**
   Executes action in Playwright
5. **LoadingPage**
   Waits for Angular + network idle
6. **Screenshot + Report**
   Captured per step, saved to .docx

Key design principle:

StepProcessor is the only class that knows all handlers exist.
Every handler is independently testable and replaceable.

THE SOLUTION

Keyword-Driven Test Automation via Excel.

Testers write steps in plain English.
The framework handles everything else.

STEP	DESCRIPTION
1	Login to CRM.
2	Go to Business CRM / Individuals.
3	Insert 12000017095 in Customer No.
4	Press Search.
5	View 360.
6	Logout from CRM.

TECH STACK

Built with the right tools.

- **Playwright**
  Browser automation — fast & reliable
- **Java 11**
  Type-safe, production-grade language
- **TestNG**
  Test runner & suite management
- **Apache POI**
  Excel reading + Word report writing
- **SQL Server**
  DB-driven navigation & validation

HOW IT WORKS

From Excel row to Word report in 4 steps.

1. **Excel File**
   Test cases in plain English

2. **Step Processor**
   Keywords route to 19+ handlers

3. **Playwright Engine**
   Real browser execution + screenshots

4. **Word Report**
   Timestamped .docx with every step

THE KEYWORD ENGINE

One step string. Infinite actions.

19+ handler classes cover every UI interaction:

Go to · Press · Insert · Select · Upload · Export · Drill down · Drag · In Group · Enable · View 360 · Wait WO

Each handler:

- Locates UI elements by text / attribute / XPath
- Waits for Angular overlays to clear
- Captures a screenshot after each action

AUTOMATED REPORTS

Every run produces evidence.

Word Report (.docx)
Per test case. Steps + embedded screenshots. Zero manual effort.

Performance Report (.docx)
Full suite timing. Slowest steps highlighted in amber. Pass/Fail cards.

Validation Excel (.xlsx)
TC IDs with DB-backed Pass/Fail results written back automatically.

PERFORMANCE REPORT — DEEP DIVE

Know exactly where time is wasted.

#	Step	Duration	Status
1	Login to CRM	1s 204ms	OK
2	Go to Business CRM	2s 891ms	OK
3	Insert Customer No.	4s 102ms	OK
4	Press Search	7s 340ms	SLOW
5	View 360	1s 050ms	OK
6	Logout	0s 812ms	FAIL

Amber = slow step (> 5s)
Red = failed step
Steps timed individually to the millisecond.

WHAT I LEARNED

6 lessons from building this.

Angular is tricky

waitForNetworkIdle isn't enough — ng-show overlays need custom wait logic.

Design for non-coders

If testers can't write steps themselves, the framework has failed its own goal.

Naming matters

Keyword matching breaks when step text is ambiguous. Strict conventions are critical.

Reports build trust

Stakeholders who can't read code can read a Word doc with screenshots.

Time everything

You can't optimize what you don't measure. The perf report proved its value day one.

DB is your friend

Driving navigation from a DB table means zero code changes when menus shift.

OPEN TO WORK

Let's connect.

If you're building a QA team or looking for
a test automation engineer — let's talk.

GitHub
Full source code

LinkedIn
DM is open

README
Detailed docs

playwright-uitesting
Keyword-Driven UI Automation Framework

#UIAutomation #Playwright #Java #QA #TestAutomation #OpenToWork